

Programação Orientada a Objetos

PROJETO DE STRINGS

em

C++

Introdução

- Uma classe pode **alocar memória**
 - Como vimos no exemplo:

```
class String
{
private:
    char * str;           // ponteiro para string
    size_t tam;           // tamanho da string
    static int Objs;       // número de instâncias

public:
    String();              // construtor padrão
    ~String();             // destrutor
    String(const char s[]); // construtor
    String(const String& s); // construtor de cópia

    String & operator=(const String & s); // operador de atribuição
};
```

Introdução

- A **alocação de memória** requer alguns cuidados
 - Realização de cópia profunda

Construtor de cópia

Inicialização e passagem de parâmetros por valor

```
String lang { "C++" }; // construção
String prog { lang };  // inicialização
Valor(prog);           // passagem por valor
```

Operador de atribuição

Usado na atribuição entre objetos da mesma classe

```
String lang { "C++" }; // construtor
String prog;           // construtor padrão
prog = lang;           // operador de atribuição
```

Introdução

- Estamos prontos para **expandir a classe String**
 - Tornando-a mais próxima de `std::string`
 - Oportunidade de aprendizado

```
friend bool operator<(const String& s1, const String& s2);  
friend bool operator>(const String& s1, const String& s2);  
friend bool operator==(const String& s1, const String& s2);
```

COMPARAÇÕES

```
friend istream& operator>>(istream& os, const String& s);
```

LEITURA

```
char & operator[](int i);  
const char & operator[](int i) const;
```

ACESSO

```
int Size() const;  
static int Count();
```

DIVERSOS

Construtor Padrão

- Cria uma **String** com **um caractere**

- O caractere nulo (**\0**)

```
String::String()  
{  
    tam = 0;  
    str = new char[1] { '\0' };  
    Obj++;  
}
```

- Por que usar [] se é apenas um caractere?

```
str = new char { '\0' };
```

```
delete [] str; ❌
```

Construtor Padrão

- Precisamos **manter compatibilidade**

- O destrutor usa `delete [ ]`

- Devemos usar `new [ ]`

```
str = new char[1] { '\0' };
```

- Poderíamos usar `nullptr`[†]

```
String::String()  
{  
    tam = 0;  
    str = nullptr;  
    Objs++;  
}
```

[†] Não podemos acessar o endereço zero (null)

```
char ideia[15] = "ideia ruim";
```

```
char * p1;  
char * p2 = ideia;  
char * p3 = new char;  
char * p4 = nullptr;
```

```
delete [ ] p1; // comportamento indefinido  
delete [ ] p2; // comportamento indefinido  
delete [ ] p3; // comportamento indefinido  
delete [ ] p4; // ok
```

Comparações

- Para fazer as **funções de comparação**
 - Podemos usar strcmp de **<cstring>**

```
int strcmp (const char * s1, const char * s2);
```

= 0 : string iguais

< 0 : primeiro argumento é menor

> 0 : primeiro argumento é maior

```
bool operator<(const String& s1, const String& s2)
{
    return (strcmp(s1.str, s2.str) < 0)
}
```

Comparações

- A **comparação de maior** pode usar a de menor
 - E seria uma boa candidata a função *inline*

```
bool operator>(const String& s1, const String& s2)
{
    return s2 < s1;
}
```

- A **comparação de igual** usa strcmp

```
bool operator==(const String& s1, const String& s2)
{
    return (strcmp(s1.str, s2.str) == 0)
}
```


Comparações

- As comparações serão **funções amigas**
 - Permitem a comparação com **char[]**

```
class String
{
    ...
```

```
    friend bool operator==(const String& s1, const String& s2);
};
```

```
if ( "C++" == lang )
```

```
if ( operator==( "C++", lang ) )
```

```
if ( operator==( String("C++"), lang ) )
```

String(const char s[]);

Construtor com um parâmetro
funciona como função de
conversão

Comparações

- As comparações serão **funções amigas**

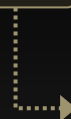
- Métodos não seriam melhores?

```
class String
{
    ...

    bool operator==(const String& s); // método da classe
};
```

`if ( lang == "C++" )`→ `if ( lang.operator=="C++" )`

`if ( "C++" == lang )`→ `if ( "C++".operator==(lang) )`



Não é um objeto
da classe String

Acesso a Caracteres

- O **acesso a caracteres** da String pode ser obtido
 - Sobrecarregando o operador []

```
String pet { "Gato" };  
cout << pet[0];  
  
      ↓  
cout << pet.operator[](0);  
  
                        ↓  
char & String::operator[](int i)  
{  
    return str[i];  
};
```

```
class String  
{  
private:  
    char * str;  
    size_t tam;  
    static int objs;  
  
    ...  
};
```

Acesso a Caracteres

- Retornar uma **referência** permite atribuições

```
char & String::operator[](int i)
{
    return str[i];
};
```

```
String pet { "Pato" };
cout << pet[0];
```

```
pet[0] = 'G';
```

Referência
para str[0]



```
class String
{
private:
    char * str;
    size_t tam;
    static int objs;

    ...
};
```

Acesso a Caracteres

- E se a **String** é uma **constante**?

```
const String pet { "Cachorro" };  
cout << pet[0];
```

↓

```
cout << pet.operator[] (0);
```



error C2678: binary '[': no operator found which takes a left-hand operand of type 'const String' (or there is no acceptable conversion)

- Vamos ter um erro de compilação:
 - O **operador** pode alterar o objeto

Acesso a Caracteres

- C++ distingue entre **métodos const e não-const**
 - É possível fornecer uma **segunda versão**
 - Utilizada apenas para const String

O retorno é uma referência constante:
não pode ser usado para mudar o objeto

```
const char & String::operator[](int i) const
{
    return str[i];
};
```

Método constante:
indica que não vai
alterar o objeto

Acesso a Caracteres

- As duas **versões** do método podem **coexistir**

```
char & String::operator[](int i)      { return str[i]; };  
const char & String::operator[](int i) const { return str[i]; };
```

- O compilador decide pelo **tipo do objeto**

```
String txt { "Volto já" };  
const String msg { "Volto logo" };
```

```
cout << txt[0];  ✓      // usa versão não constante  
cout << msg[0];  ✓      // usa versão constante
```

```
txt[4] = 'a';    ✓      // usa versão não constante  
msg[4] = 'a';    ✗      // erro de compilação
```

Métodos Estáticos

- O acesso a **membros estáticos**:
 - Pode ser feito por **métodos estáticos**
 - Não dependem de objetos

```
class String
{
    private:
        static int Objs;
        ...

    public:
        static int Count();
};
```

Usado apenas
na declaração

```
int String::Count()
{
    return Objs;
};
```

```
int main()
{
    int instances = String::Count();
}
```

Método estático
da classe String

Métodos Estáticos

- Só podem acessar membros estáticos
 - Não podem acessar atributos

```
class String
{
private:
    char * str;
    size_t tam;
    static int Objs;
    ...

public:
    static int Count();
};
```

```
int String::Count()
{
    // str e tam não ficam visíveis
    // para métodos estáticos
    int valor = tam;
};
```



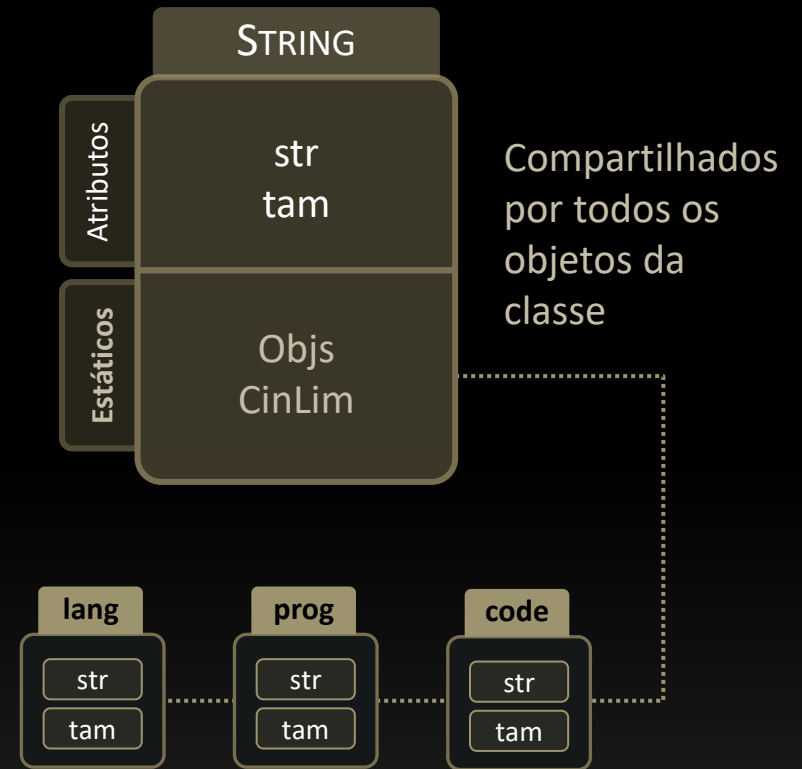
error C2597: illegal reference to non-static member
'String::tam'

Métodos Estáticos

- São úteis para **ajustar flags**
 - Controlar o comportamento da classe:
 - Formatação, tamanhos, limites etc.

```
class String
{
private:
    static int Objs;
    static int CinLim;
    ...

public:
    static int Count();
    static void SetLimit(int lim);
};
```



Leitura de Strings

- Podemos ler Strings com cin
 - Sobrecarregando o operador >>

```
istream & operator>>(istream & is, String & s)
{
    char * temp = new char[String::CinLim];
    is.getline(temp, String::CinLim);

    s = temp;
    delete [] temp;
    return is;
};
```

Constrói String temporária e realiza uma cópia dela na atribuição

s.operator=(String(temp));

```
String nome;
cin >> nome;

↓

operator>>(cin, nome);
```

```
String & operator=(const String & s)
{
    if (this == &s)
        return *this;

    delete [] str;
    tam = s.tam;
    str = new char[tam + 1];
    strcpy(str, s.str);

    return *this;
}
```

Leitura de Strings

- Podemos **otimizar** a atribuição de char []
 - Criando uma nova sobrecarga

```
String & String::operator=(const char s[])  
{  
    delete [] str;  
    tam = strlen(s);  
    str = new char[tam + 1];  
    strcpy(str, s);  
  
    return *this;  
}
```

```
String msg;  
char temp[] = "Olá";  
msg = temp;
```

```
msg.operator=(temp);
```

Não constrói uma String temporária porque existe um método apropriado

Resumo

- Classes podem replicar tipos básicos do C++
 - Usando os mesmos operadores
 - Atribuição =
 - Acesso a elemento []
 - Leitura >>
 - Exibição <<
 - Comparação == > <
 - Métodos estáticos só acessam membros estáticos
 - Métodos não estáticos acessam todos os membros